Hacettepe University — Computer Engineering
CMP 720 EMBEDDED SYSTEMS DESIGN

Spring 2026 — Mini In-Class Project Series Part 2

Lab 2: Platform Bring-Up & Architecture Mapping

SafePulse — Embedded Patient Monitoring System

Week 3: Embedded Hardware & System Architecture

Duration:	1 – 1.5 hours
Tools Required:	Renode, ARM GCC toolchain, terminal, Week 2 handout
Deliverable:	Working firmware running in Renode with annotated architecture mapping
Prerequisites:	Completed Lab 1 (Week 2) template

You will need to look things up during this lab. All documents are freely available online.

STM32F4 Reference Manual (RM0090) — *the primary reference for everything hardware-related.*
The sections you need today are:

- **Section 2.3** — Memory map (base addresses of all peripherals; this is where `.repl` addresses come from)
- **Section 7** — Reset and Clock Control (RCC); register map at end of section lists every clock-enable bit
- **Section 8** — General-purpose I/O (GPIO); MODER and ODR register layouts
- **Section 30** — USART; CR1, BRR, and SR register descriptions

Renode documentation — `https://renode.readthedocs.io`
For `.repl`/`.resc` syntax reference if you are curious, but the provided files are complete — you do not need to modify them today.

Sensor datasheets (for context only — not needed to complete today's tasks): MAX30102 datasheet (HR/SpO_2); TMP102 datasheet (temperature). Device I2C addresses are defined in these documents and will be used in Week 4.

1 Overview

This week, you transition from paper design to a running simulation. You will set up the STM32F4 platform in **Renode**, load and run firmware, and establish the mapping between your Week 2 block diagram and the actual simulated hardware. By the end of this session, you will have a bare-metal program that initializes UART, blinks an LED via GPIO, and prints sensor-like data over serial — all running inside Renode.

> **What is Renode?** Renode is an open-source embedded systems simulation framework developed by Antmicro. Unlike software-only emulators, Renode simulates the actual hardware peripherals (UART, I2C, GPIO, timers, etc.) so that your firmware binary runs exactly as it would on real silicon — but without needing a physical board.

1.1 Learning Objectives

1. Understand Renode's platform description (`.repl`) and script (`.resc`) file formats.
2. Map an abstract block diagram (from Week 2) to concrete simulated peripherals.
3. Write bare-metal C code that initializes and uses UART and GPIO on STM32F4.
4. Successfully compile, load, and run firmware inside Renode.
5. Identify why a delay-loop approach is inadequate for real-time systems (motivation for Week 4).

2 Provided Files

You are provided with a project folder containing the following files:

File	Purpose
`stm32f4.repl`	Renode platform description — defines the MCU and its peripherals
`safepulse.resc`	Renode script — loads the platform, binary, and configures analyzers
`main.c`	Skeleton bare-metal firmware with TODOs for you to complete
`startup.s`	ARM Cortex-M4 startup assembly (vector table, stack init) — *do not modify*
`stm32f4.ld`	Linker script for STM32F4 memory layout — *do not modify*
`Makefile`	Build configuration for ARM GCC cross-compilation

3 Understanding the Renode Platform

3.1 The Platform Description (`.repl`)

The `.repl` file describes the hardware topology to Renode. It is a **human-authored** file — Renode does not auto-generate it. Every base address you see (e.g., `0x40011000` for UART1) comes directly from the STM32F4 Reference Manual's memory map table (RM0090, Section 2.3). When you write a `.repl` for a new MCU, you look up each peripheral's base address in the manufacturer's reference manual and enter it here. Renode trusts these values and places the simulated peripheral at exactly that address in the simulated address space.

How to read a .repl entry: The line `uart1: UART.STM32_UART @ sysbus <0x40011000, +0x100>` means: "create a peripheral named `uart1`, using Renode's built-in `STM32_UART` model, mapped onto the system bus starting at address `0x40011000` and occupying `0x100` (256) bytes." The `-> nvic@37` line means its interrupt line connects to NVIC input 37 – also from the datasheet's interrupt table. The `+0x100` range is the address-decode window: accesses from `0x40011000` to `0x400110FF` go to this peripheral. RM0090 Section 30 shows the USART register offsets only reach around `0x1C`, but ARM peripherals are typically allocated power-of-two blocks, so most offsets in that 256-byte window are unused.

Examine the provided `stm32f4.repl`:

Listing 1: stm32f4.repl — Platform Description (provided; addresses from RM0090 Section 2.3)

```
// SafePulse STM32F4 Platform Description for Renode
// Lab 2: Platform Bring-Up & Architecture Mapping

// UART.STM32_UART is Renode's internal model class name; it's what
   Renode's developers chose to call their simulation model. The
   address is what matters.
uart1: UART.STM32_UART @ sysbus <0x40011000, +0x100>
    -> nvic@37

gpioA: GPIOPort.STM32_GPIOPort @ sysbus <0x40020000, +0x400>
    numberOfAFs: 16
    [0-15] -> exti@[0-15]

gpioD: GPIOPort.STM32_GPIOPort @ sysbus <0x40020C00, +0x400>
    numberOfAFs: 16
    [0-15] -> exti@[0-15]

i2c1: I2C.STM32F4_I2C @ sysbus <0x40005400, +0x400>
    EventInterrupt -> nvic@31
    ErrorInterrupt -> nvic@32

timer2: Timers.STM32_Timer @ sysbus <0x40000000, +0x400>
    frequency: 84000000   // APB1 timer input clock (Hz) -- NOT the
       sampling rate
    initialLimit: 0xFFFFFFFF
    -> nvic@28

rcc: Miscellaneous.STM32F4_RCC @ sysbus 0x40023800
    rtcPeripheral: rtc

rtc: Timers.STM32F4_RTC @ sysbus 0x40002800

nvic: IRQControllers.NVIC @ sysbus 0xE000E000
    priorityMask: 0xF0
    systickFrequency: 84000000
    IRQ -> cpu@0

cpu: CPU.CortexM @ sysbus
    cpuType: "cortex-m4f"
    nvic: nvic
```

```
exti: IRQControllers.STM32F4_EXTI @ sysbus 0x40013C00
    numberOfOutputLines: 24
    [0-4] -> nvic@[6-10]

sram: Memory.MappedMemory @ sysbus 0x20000000
    size: 0x20000

flash: Memory.MappedMemory @ sysbus 0x08000000
    size: 0x100000
```

Note on the used UART: The STM32F4 peripheral we will use is actually a USART (Universal Synchronous/Asynchronous Receiver-Transmitter). It supports both synchronous (clocked) and asynchronous modes. When you use it in asynchronous mode (no clock line, just TX/RX), it behaves as a plain UART.
Note on frequency: 84000000 for TIM2. This tells Renode that TIM2's *input clock* runs at 84 MHz — it is the rate at which the timer's internal counter ticks, not the sampling rate. To produce a 200 ms interrupt, you configure the timer's prescaler and auto-reload registers so the counter overflows every 200 ms. For example: prescaler = 8399 gives a 10 kHz tick; auto-reload = 1999 then fires every 200 ms. We will configure this in Week 4. The 84 MHz figure itself comes from the STM32F4's PLL configuration (see RM0090 Section 7, RCC clock tree).

3.2 Task: Architecture Mapping

Using your Week 2 block diagram and the `.repl` listing above, annotate the mapping between your abstract design blocks and the concrete Renode peripherals. For each row, find the matching peripheral in the `.repl`, record its base address, and note how it is connected.

Quick I2C primer: The I2C protocol allows multiple devices to share a single two-wire bus (SDA + SCL). Each device has a unique **7-bit device address** burned in at manufacture (or set via address pins). The MCU (master) sends the target address at the start of every transaction; only the device with that address responds. This is how a single `i2c1` peripheral in the `.repl` can communicate with both the heart rate sensor and the temperature sensor — they share the bus but have different addresses. The specific addresses for each sensor (e.g., 0x57 for a MAX30102, 0x48 for a TMP102) are found in the respective **sensor datasheets**; we will assign them when adding the simulated sensors in Week 4.

Week 2 Block	Renode Peripheral	Base Address	Notes
[Heart Rate / SpO2 Sensor]	*[Which peripheral?]*	*[Address?]*	*[How connected?]*
[Body Temp Sensor]	*[Which peripheral?]*	*[Address?]*	*[How connected?]*
[Alarm Indicator]	*[Which peripheral?]*	*[Address?]*	*[Which pin?]*
[Nurse Station Display]	*[Which peripheral?]*	*[Address?]*	*[Baud rate?]*
[Sampling Timer]	*[Which peripheral?]*	*[Address?]*	*[Input clock freq?]*

3.3 The Renode Script (`.resc`)

The `.resc` script tells Renode how to set up and start the simulation:

Listing 2: safepulse.resc — Renode Launch Script (provided)

```
# SafePulse Lab 2 - Renode Script
# Run with: renode safepulse.resc

using sysbus

# Create and configure the machine
mach create "SafePulse"
machine LoadPlatformDescription @stm32f4.repl

# Load the compiled firmware binary
sysbus LoadELF @build/safepulse.elf

# Open UART analyzer to see serial output
showAnalyzer uart1

# Set log level for debugging
logLevel -1 uart1

# Start the simulation
start
```

4 Skeleton Code — Complete the TODOs

The provided `main.c` contains a working structure with critical sections left for you to implement. Read through the complete file, then fill in the TODOs.

How to approach the TODOs. Each TODO comment tells you *what* to do (which register to set, which bit to use). The register names and bit positions were looked up in the **STM32F4 Reference Manual (RM0090)**. For each TODO, the relevant RM0090 section is noted in the comment. If you want to understand *why* a particular bit or value is used, open RM0090 to that section and read the register description — this is exactly how embedded firmware is written in practice.

The `#define` macros at the top of `main.c` translate raw addresses and offsets into readable names. For example, `RCC_APB2ENR` expands to a volatile pointer dereference of address `0x40023800 + 0x44` — the exact address of that register in the STM32F4 (verifiable in RM0090 Section 7 register map).

The BASE, CR1, BRR, DR, and SR registers are needed because: BASE (‘`0x40011000`‘) is the starting memory address of the USART1 peripheral — all other register addresses are calculated as offsets from this base. CR1 (Control Register 1) enables the peripheral and its transmitter. BRR (Baud Rate Register) sets the communication speed. SR (Status Register) lets you check hardware flags, such as whether the transmit buffer is empty before sending the next byte. DR (Data Register) is where you actually write the byte you want to send — writing a character to DR causes the UART hardware to serialize it and shift it out on the TX line. Together, these are the minimum registers needed to get serial output working.

Listing 3: main.c — Skeleton (complete the TODO sections)

```c
#include <stdint.h>

/* ============================================================
 * Register Definitions for STM32F4
 *
 * All base addresses and offsets are from the STM32F4
 * Reference Manual (RM0090):
 *   - RCC registers: Section 7 (register map at end of section)
 *   - GPIO registers: Section 8 (register map at end of section)
 *   - USART registers: Section 30 (register map at end of section)
 * ============================================================ */

/* Reset and Clock Control (RCC) -- RM0090 Section 7 */
#define RCC_BASE        0x40023800
#define RCC_AHB1ENR     (*(volatile uint32_t *)(RCC_BASE + 0x30))
#define RCC_APB2ENR     (*(volatile uint32_t *)(RCC_BASE + 0x44))

/* GPIO Port D (LEDs on STM32F4-Discovery) -- RM0090 Section 8 */
#define GPIOD_BASE      0x40020C00
#define GPIOD_MODER     (*(volatile uint32_t *)(GPIOD_BASE + 0x00))
#define GPIOD_ODR       (*(volatile uint32_t *)(GPIOD_BASE + 0x14))

/* USART1 -- RM0090 Section 30 */
#define USART1_BASE     0x40011000
#define USART1_SR       (*(volatile uint32_t *)(USART1_BASE + 0x00))
```

```c
#define USART1_DR       (*(volatile uint32_t *)(USART1_BASE + 0x04))
#define USART1_BRR      (*(volatile uint32_t *)(USART1_BASE + 0x08))
#define USART1_CR1      (*(volatile uint32_t *)(USART1_BASE + 0x0C))

/* Bit definitions -- bit positions from RM0090 register descriptions */
/* See FAQ for more clarifications */
#define USART_SR_TXE    (1 << 7)   /* Transmit data register empty */
#define USART_CR1_UE    (1 << 13)  /* USART enable */
#define USART_CR1_TE    (1 << 3)   /* Transmitter enable */

#define LED_GREEN_PIN   12  /* PD12 = Green LED on STM32F4-Discovery */

/* ============================================================
 * Helper Functions (provided)
 * ============================================================ */

/* Simple busy-wait delay (NOT suitable for real-time!) */
void delay_ms(uint32_t ms) {
    for (volatile uint32_t i = 0; i < ms * 8400; i++) {
        __asm__("nop");
    }
}

/* Send a single character over UART */
void uart_putc(char c) {
    while (!(USART1_SR & USART_SR_TXE));  /* Wait until TX empty */
    USART1_DR = c;
}

/* Send a string over UART */
void uart_puts(const char *s) {
    while (*s) {
        uart_putc(*s++);
    }
}

/* Convert integer to decimal string (simple implementation) */
void uart_put_int(int val) {
    char buf[12];
    int i = 0;
    if (val < 0) { uart_putc('-'); val = -val; }
    if (val == 0) { uart_putc('0'); return; }
    while (val > 0) { buf[i++] = '0' + (val % 10); val /= 10; }
    while (i > 0) { uart_putc(buf[--i]); }
}

/* ============================================================
 * TODO 1: Initialize UART1
 *
 * Reference: RM0090 Section 30 (USART) -- register map at end
 *            RM0090 Section 7  (RCC)   -- APB2ENR register
 *
 * Steps:
 *   a) Enable USART1 clock via RCC_APB2ENR (bit 4).
```

```c
 *       Look up "RCC_APB2ENR" in RM0090 Section 7 to verify
 *       which bit corresponds to USART1EN.
 *
 *    b) Set baud rate register (BRR).
 *       For a peripheral clock of 84 MHz and 115200 baud:
 *         BRR = 84,000,000 / 115,200 = 729 = 0x2D9
 *       (See RM0090 Section 30.3.4 for the BRR formula.)
 *
 *    c) Enable the USART and its transmitter by writing to CR1.
 *       You need two bits set: UE (bit 13) enables the whole
 *       USART, and TE (bit 3) enables the transmitter.
 *       To set multiple bits in one write, combine masks with
 *       the bitwise OR operator (|). For example:
 *         USART_CR1_UE = (1 << 13) = 0x2000
 *         USART_CR1_TE = (1 << 3)  = 0x0008
 *         USART_CR1_UE | USART_CR1_TE = 0x2008
 *       Write the combined value to USART1_CR1.
 *       Note: Use direct assignment (=) rather than |= so CR1
 *       contains only these bits (others stay 0). This selects
 *       default settings: 8 data bits, no parity
 *       (see RM0090 Section 30.6.4).
 * ============================================================ */
void uart_init(void) {
    // TODO: Your code here
}

/* ============================================================
 * TODO 2: Initialize GPIO for LED
 *
 * Reference: RM0090 Section 8 (GPIO) -- MODER and ODR registers
 *            RM0090 Section 7 (RCC)  -- AHB1ENR register
 *
 * Steps:
 *    a) Enable GPIOD clock via RCC_AHB1ENR (bit 3).
 *       Look up "RCC_AHB1ENR" in RM0090 Section 7 to verify
 *       which bit corresponds to GPIODEN.
 *
 *    b) Configure PD12 as a general-purpose output in GPIOD_MODER.
 *       Background (RM0090 Section 8.4.1): MODER uses 2 bits per pin.
 *       Pin 0 uses bits [1:0], pin 1 uses [3:2], ... pin N uses
 *       bits [2N+1:2N]. Modes: 00=input, 01=output, 10=AF, 11=analog.
 *       For PD12, bits are [25:24] since 12 * 2 = 24. Write 01 there.
 *
 *       Use a clear-then-set pattern:
 *       Step 1 - Clear bits [25:24] to 00
 *       Step 2 - Set output mode (01):
 *
 *       Why two steps? If the field was 11 (analog), |= 01 would
 *       keep it 11. Clearing first guarantees the correct value
 *       for any prior state.
 * ============================================================ */
void led_init(void) {
    // TODO: Your code here
}
```

```c

/* ============================================================
 * TODO 3: Toggle LED
 *
 * Reference: RM0090 Section 8 (GPIO) -- ODR register
 *
 * XOR the bit for PD12 in GPIOD_ODR to toggle the pin.
 * XOR is the standard read-modify-write pattern for toggling
 * a single bit without affecting others.
 * ============================================================ */
void led_toggle(void) {
    // TODO: Your code here
}

/* ============================================================
 * Main Function
 * ============================================================ */
int main(void) {
    /* Initialize peripherals */
    uart_init();
    led_init();

    /* Startup banner */
    uart_puts("\r\n");
    uart_puts("========================================\r\n");
    uart_puts("  SafePulse v0.1 - System Online\r\n");
    uart_puts("  Patient Vital Signs Monitor\r\n");
    uart_puts("========================================\r\n");
    uart_puts("\r\n");

    /* Simulated patient data (hardcoded for this week) */
    int heart_rate = 72;      /* BPM */
    int spo2 = 97;            /* percent */
    int temperature = 368;    /* 36.8 C (x10 for fixed-point) */

    int count = 0;

    /* Main loop: blink LED and print data at ~1Hz */
    while (1) {
        led_toggle();

        /* Print current "readings" */
        uart_puts("[");
        uart_put_int(count);
        uart_puts("] HR=");
        uart_put_int(heart_rate);
        uart_puts(" bpm, SpO2=");
        uart_put_int(spo2);
        uart_puts("%, Temp=");
        uart_put_int(temperature / 10);
        uart_puts(".");
        uart_put_int(temperature % 10);
        uart_puts(" C, State=NORMAL\r\n");

```

```
        count++;

        /* Delay ~1 second using busy-wait */
        delay_ms(1000);
    }

    return 0;  /* Never reached */
}
```

5 Implementation Tasks

Complete the following tasks in order. For each, fill in the code and verify the result in Renode.

5.1 Task 1: Implement `uart_init()`

Enable the USART1 clock, configure the baud rate, and enable the transmitter. The TODO comment in `main.c` gives you the bit numbers and formula; look them up in RM0090 Sections 7 and 30 if you want to understand why those values are used.

5.2 Task 2: Implement `led_init()`

Enable the GPIOD clock and configure PD12 as a general-purpose output pin. Refer to RM0090 Section 8 for the MODER register bit layout.

5.3 Task 3: Implement `led_toggle()`

Toggle the LED output pin using XOR on the output data register. Refer to RM0090 Section 8 for the ODR register.

5.4 Task 4: Build and Run

1. Compile the firmware:

```
$ make clean && make
```

2. Launch Renode and load the script:

```
$ renode safepulse.resc
```

3. Verify that:

a) The UART analyzer window shows the startup banner "SafePulse v0.1 — System Online"

b) Sensor readings are printed every ∼1 second with incrementing count

c) The GPIOD output register toggles (observable via Renode monitor or log). Some Renode installations do not provide a GPIO analyzer backend for `STM32_GPIOPort` (e.g., showAnalyzer gpioD may fail with a "No backend found..." error). In this case, verify the LED/GPIOD toggle directly via the Renode Monitor by repeatedly reading the GPIOD Output Data Register (ODR). Run the following command in Renode Monitor to read `GPIOD_ODR`:

```
sysbus ReadDoubleWord 0x40020C14
```

Build troubleshooting: If you get linker errors, ensure `arm-none-eabi-gcc` is in your PATH. If Renode can't find the ELF file, check that the build output is at `build/safepulse.elf`. Use `make V=1` for verbose build output.

6 Provided Build System

For reference, here is the `Makefile` provided with the project:

Listing 4: Makefile (provided — do not modify)

```
# SafePulse Lab 2 - Makefile
CC = arm-none-eabi-gcc
OBJCOPY = arm-none-eabi-objcopy

CFLAGS = -mcpu=cortex-m4 -mthumb -mfloat-abi=soft
CFLAGS += -nostdlib -nostartfiles
CFLAGS += -Wall -Wextra -O1 -g
CFLAGS += -T stm32f4.ld

SRC = startup.s main.c
TARGET = build/safepulse

all: $(TARGET).elf

$(TARGET).elf: $(SRC)
        @mkdir -p build
        $(CC) $(CFLAGS) $(SRC) -o $(TARGET).elf

$(TARGET).bin: $(TARGET).elf
        $(OBJCOPY) -O binary $(TARGET).elf $(TARGET).bin

clean:
        rm -rf build/

.PHONY: all clean
```

7 Observation & Reflection

After your firmware is running successfully, answer the following questions:

Q1: Look at the `delay_ms()` function. It uses a busy-wait loop. What percentage of CPU time is spent doing "useful work" (printing, toggling LED) versus waiting? Estimate roughly.

[Your answer]

Q2: Suppose a critical SpO2 event occurs 10 ms after the main loop enters `delay_ms(1000)`. How long would it take for the system to detect and respond? How does this compare to your NFR-01 deadline of 100 ms?

[Your answer]

Q3: In the `.repl` file, both the heart rate sensor and temperature sensor will connect to `i2c1`. How does a single I2C bus distinguish between multiple devices? What would you need to define for each sensor?

[Your answer]

8 FAQ

Why the startup assembly (vector table, stack init)?
On ARM Cortex-M4 reset, hardware only reads the initial stack pointer from address `0x00000000` (mapped to flash at `0x08000000`) and the reset handler from `0x00000004`, then jumps there. It does not know about `main()`. The `startup.s` file provides (1) the vector table: an array of pointers where entry 0 is the initial stack pointer and entry 1 is the reset handler, followed by exception/interrupt handlers; and (2) the reset handler itself, which zeros `.bss`, copies initialized data from flash to RAM, and then calls `main()`. The linker script (`stm32f4.ld`) places this table at the correct flash address. Without this bootstrap, C code cannot run.

Why are RCC, RTC, and NVIC in the `.repl`?
RCC: The Reset and Clock Control peripheral is the "power switch" for all other peripherals. Clocks are gated off by default, so your firmware must enable them by writing RCC registers. Renode needs an RCC model so those writes actually turn peripherals on; otherwise clock enables would do nothing.
RTC: Renode's `STM32F4_RCC` model manages the RTC clock source, so it expects an RTC device to exist even if you do not use it in this lab.
NVIC: The NVIC is the Cortex-M4 interrupt manager. Lines like `-> nvic@37` wire peripheral

interrupts to specific NVIC inputs. Even if you do not use interrupts in Lab 2, the wiring must be valid for a well-formed simulation, and you will use it in Week 4 for timer interrupts.

NVIC at `0xE000E000` — where does this address come from?

It is not in the STM32F4 reference manual because the NVIC is part of the ARM Cortex-M4 core, not an ST peripheral. ARM defines a fixed System Control Space (SCS) at `0xE000E000--0xE000EFFF` that contains the NVIC, SysTick, and System Control Block. This address is specified in the ARMv7-M Architecture Reference Manual and the Cortex-M4 TRM, and it is the same on all Cortex-M processors across vendors.

What do the volatile pointer macros do, and why?

Example: `#define RCC_BASE 0x40023800` and `#define RCC_AHB1ENR (*(volatile uint32_t *)(RCC_BASE + 0x30))`.
Breakdown: `RCC_BASE + 0x30 = 0x40023830` is the AHB1ENR register address (RM0090 Section 7). `(uint32_t *)` casts that integer to a pointer to a 32-bit word. `volatile` tells the compiler this memory can change outside your code. `*` dereferences the pointer so you can read/write it like a variable. This makes `RCC_AHB1ENR |= (1 << 3);` compile into an actual write to address `0x40023830`.
Why volatile: Hardware registers change on their own (e.g., USART1_SR sets TXE when ready). Without `volatile`, the compiler may cache reads or reorder writes, and a loop like `while (!(USART1_SR & USART_SR_TXE))` could spin forever. `volatile` forces every access to hit the real memory-mapped register, which is mandatory for hardware I/O.

What is EXTI?

EXTI stands for External Interrupt/Event Controller. It is an STM32-specific peripheral (RM0090 Section 12) that manages interrupts triggered by GPIO pin changes (rising edge, falling edge, or both). When you need to detect an external event — like a button press or a sensor interrupt pin — the GPIO line connects through EXTI to the NVIC. The `[0-15] -> exti@[0-15]` lines in the GPIO definitions mean "GPIO pins 0–15 can trigger EXTI lines 0–15." Then `[0-4] -> nvic@[6-10]` in the EXTI definition routes EXTI lines 0–4 to NVIC interrupt inputs 6–10. It is in the `.repl` as part of the standard STM32F4 interrupt plumbing, and it becomes relevant for pin-change interrupts (e.g., a sensor data-ready pin).

Why 84 MHz for TIM2?

This comes from the STM32F4 clock tree: the board's 8 MHz external crystal is multiplied by the PLL to 168 MHz for HCLK. APB1 runs at HCLK/4 = 42 MHz, but when the APB1 prescaler is not 1, the timer clocks are doubled, so TIM2 sees 42 MHz × 2 = 84 MHz (RM0090 Section 7). On real hardware you can change this by reconfiguring the PLL or APB prescalers. In Renode, the `frequency: 84000000` parameter directly sets the simulated tick rate. For this lab, 84 MHz matches the standard STM32F4-Discovery configuration, so Renode behavior aligns with hardware.

Why include GPIOA if the LED is on GPIOD?

GPIOA is included because USART1 TX/RX are on PA9/PA10. Even though this lab does not explicitly configure GPIOA alternate-function registers (Renode's UART model works without that

step), having GPIOA in the `.repl` makes the platform description complete and realistic. It also supports future labs or projects where GPIOA pins are used for SPI, additional UARTs, etc. On real hardware, you would set PA9 to USART1_TX and PA10 to USART1_RX using GPIOA's MODER and AFRH registers; Renode is more forgiving, but including GPIOA keeps the platform description honest.

What does `logLevel -1 uart1` do?

This Renode monitor command sets logging verbosity for the `uart1` peripheral to level `-1` (Noisy, the most verbose). Renode log levels are: `-1` = Noisy (all events, including register accesses), `0` = Debug, `1` = Info, `2` = Warning, `3` = Error. With `-1`, Renode logs every USART1 register read/write, which is useful for debugging UART issues (e.g., seeing BRR/CR1 writes and each data-register write).

Why `LED_GREEN_PIN` 12?

This is a board-level design choice on the STM32F4-Discovery. ST wired the four LEDs to GPIOD: green on PD12, orange on PD13, red on PD14, and blue on PD15. The "12" means pin 12 of Port D. This mapping is documented in the STM32F4-Discovery user manual (UM1472), not the MCU reference manual (RM0090), because RM0090 covers the chip itself while UM1472 covers the board wiring. Any GPIO could drive an LED, but ST chose PD12 for green on this board.

How do we know the shift values for bit definitions?

They come directly from the register bit-field diagrams in RM0090. The manual shows which bit position corresponds to each named field. For example: in USART1_SR (RM0090 Section 30.6.1), bit 7 is TXE, so `(1 << 7)` masks TXE. In USART1_CR1 (RM0090 Section 30.6.4), bit 13 is UE and bit 3 is TE, so `(1 << 13)` and `(1 << 3)`. The pattern `(1 << n)` is the standard C idiom for "a value with only bit n set," which is why embedded code maps directly from the reference manual's bit positions to shift values. It can help to show one register diagram and walk through the mapping once so the pattern clicks.

9 Looking Ahead — Week 4 Preview

You now have a working bare-metal system in Renode, but it has two critical limitations that make it unsuitable for a safety-critical medical device:

1. No real sensors: You are printing hardcoded values. Next week, you will add a custom Python I2C sensor peripheral that generates realistic (and dynamically changing) patient data. The sensor addresses (0x57 for HR/SpO_2, 0x48 for temperature) will be wired up to the `i2c1` bus at that point.

2. No real-time guarantees: The busy-wait delay blocks the CPU and cannot meet your 100 ms alarm deadline. Next week, you will replace it with a **hardware timer interrupt** (TIM2) that fires every 200 ms. You will configure TIM2's prescaler and auto-reload registers to divide the 84 MHz input clock down to a 200 ms period — enabling deterministic sampling that is independent of other processing.

Keep your code! Next week's lab builds directly on today's `main.c`. You will add the timer interrupt ISR and I2C driver functions to the same file. Make sure your code compiles cleanly before leaving today.